



C Programming Basics
Lab Report and Knowledge Check

Pravina kumari Chaudhary
Bsc (Hons) Computer Science with Artificial Intelligence
Softwarica College of IT and E-commerce
ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi
20th September, 2026

TABLE OF CONTENT

Introduction.....	3
GCC Installation.....	3
C Program Structure and Examples.....	4
A. Basic Hello World Program.....	4
B. Program with User Input.....	5
C. Program with Formatted Output.....	6
Four stages of Compilation.....	7
Return values and Exit Status.....	10
Knowledge Check Questions.....	12
Program Returning integer value 50.....	15
Conclusion.....	16
GitHub Repository.....	16
Appendix.....	17

LIST OF FIGURES

Figure 1 : apt update	3
Figure 2 : gcc version check	4
Figure 3 : hello.c.....	5
Figure 4 : Compilation of hello.c.....	5
Figure 5 : input.c.....	5
Figure 6 : Program with user input.....	6
Figure 7 : userinfo.c.....	6
Figure 8 : Userinfo output.....	6
Figure 9 : Preprocessing stage.....	7
Figure 10 : Compilation Stage.....	8
Figure 11 : Assembly stage.....	9
Figure 12: Linking Stage.....	9
Figure 13: Program with return value (success.c).....	10
Figure 14: Exit status of success.c is 0 (success).....	10
Figure 15: Task 1: hello.c returns exit status 0.....	11
Figure 16 : Task 2: hello_error.c returns exit status 1.....	11
Figure 17: input.c.....	14
Figure 18 : return50.c.....	15
Figure 19 : Return value.....	16

INTRODUCTION

This is the lab report of Operating Systems, Security and Networks where the knowledge of C program structure, GCC compilation and exit status is tested. It is done since the OS is what loads programs, runs them and reads back their exit code. This shows the boundary between the code and the operating system.

GCC Installation

```
pravina@LAPTOP-6K8UDRAS:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [478 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [113 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [616 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [173 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [54.5 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [148 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [98.0 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [294 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [93.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [188 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [413 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [88.0 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3172 B]
Get:20 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Fetched 3253 kB in 3s (982 kB/s)
14 packages can be upgraded. Run 'apt list --upgradable' to see them.
pravina@LAPTOP-6K8UDRAS:~$ sudo apt install build-essentials
Error: Unable to locate package build-essentials
pravina@LAPTOP-6K8UDRAS:~$ sudo apt install build-essential
Installing:
  build-essential

Installing dependencies:
  bzip2 fakeroot g++-15 g++-x86-64-linux-gnu libalgorithm-diff-xs-perl libcrypt-dev libfakeroot libstdc++-15-dev make
  dpkg-dev g++ g++-15-x86-64-linux-gnu libalgorithm-diff-perl libalgorithm-merge-perl libdpkg-perl libfile-fcntllock-perl lto-disabled-list

Suggested packages:
  bzip2-doc debian-keyring debian-tag2upload-keyring g++-multilib g++-15-multilib gcc-15-doc bzr libstdc++-15-doc make-doc

Summary:
  Upgrading: 0, Installing: 18, Removing: 0, Not Upgrading: 14
  Download size: 19.3 MB
  Space needed: 73.4 MB / 1024 GB available
```

Figure 1 : apt update

```
pravina@LAPTOP-6K8UDRA5:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 2 : gcc version check

C Program Structure and Examples

Every C program in this lab follows the same basic structure:

- **Preprocessor directive** : the # `#include <stdio.h>` tells C to include a header file. This header lets us use input/output functions such as `printf()`. Directives do not end with a semicolon because they are not C statements.
- **main() function** : `main()` is a special function. The program starts running here. Any code inside the curly brackets `{}` will be executed.
- **printf ()** : is a function used to output (print) text to the screen.
- **return statement** : `return 0` ends the `main()` function and sends a value back to the operating system.

A. Basic Hello World Program

```
GNU nano 8.7.1                                hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Figure 3 : hello.c

This program prints Hello, World! Followed by a newline(`\n`) and returns 0 to show that it finished successfully.

```
pravina@LAPTOP-6K8UDRA5:~$ nano hello.c
pravina@LAPTOP-6K8UDRA5:~$ gcc hello.c -o hello
pravina@LAPTOP-6K8UDRA5:~$ ./hello
Hello, World!
pravina@LAPTOP-6K8UDRA5:~$
```

Figure 4 : Compilation of hello.c

B. Program with User Input

```
GNU nano 8.7.1 input.c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 5 : input.c

```
pravina@LAPTOP-6K8UDRA5:~$ nano input.c
pravina@LAPTOP-6K8UDRA5:~$ gcc input.c -o input
pravina@LAPTOP-6K8UDRA5:~$ ./input
Enter a number: 15
You entered: 15
pravina@LAPTOP-6K8UDRA5:~$
```

Figure 6 : Program with user input

The program declares an integer variable `number` and uses `scanf()` to read user input from the keyboard. The `&` operator points to the number's memory address, allowing `scanf()` to assign the input value to it. The stored integer is subsequently displayed on the console using `printf()` with the `%d` format specifier.

C. Program with Formatted Output

```
GNU nano 8.7.1                                     userinfo.c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 7 : userinfo.c

```
pravina@LAPTOP-6K8UDRA5:~$ nano userinfo.c
pravina@LAPTOP-6K8UDRA5:~$ gcc userinfo.c -o userinfo
pravina@LAPTOP-6K8UDRA5:~$ ./userinfo
Enter your name: pravina
Enter your age: 21
Enter your height (m): 1.59 m

--- User Info ---
Name: pravina
Age: 21 years
Height: 1.59 m
pravina@LAPTOP-6K8UDRA5:~$
```

Figure 8 : Userinfo output

The program demonstrates user input and formatted output across three distinct C data types: a character array (%s), an integer (%d), and a floating-point number (%f). The variable name functions directly as a memory address, eliminating the need for the & operator in its scanf() call. Additionally, the %s format specifier halts input collection at the first whitespace character (storing a single word), while %.2f formats the printed height to two decimal places.

Four stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

Stage	Command	Output	Purpose
1. Preprocessing	gcc -E hello.c -o hello.i	.i file	Expand headers and macros
2. Compile	gcc -S hello.i -o hello.s	.s file	Generate assembly code
3. Assembling	gcc -c hello.s -o hello.o	.o file	Create object file
4. Linking	gcc hello.c -o hello	executable	Final executable

Table 1 : The four stages of C compilation

1. Stage 1: Preprocessing

```
pravina@LAPTOP-6K8UDRA5:~$ gcc -E hello.c -o hello.i
pravina@LAPTOP-6K8UDRA5:~$ wc -l hello.i
844 hello.i
pravina@LAPTOP-6K8UDRA5:~$ tail -n 10 hello.i
# 977 "/usr/include/stdio.h" 3 4

# 2 "hello.c" 2

# 3 "hello.c"
int main() {
    printf("Hello, World!\n");
    return 0;
}
pravina@LAPTOP-6K8UDRA5:~$ cat hello.i | less
pravina@LAPTOP-6K8UDRA5:~$
```

Figure 9 : Preprocessing stage

Observation: hello.i has 819 lines, while hello.c has only 6, because `#include <stdio.h>` was replaced by the header file's contents. The original `main()` appears at the end of the file.

2. Stage 2: Compilation to Assembly

```
pravina@LAPTOP-6K8UDRA5:~$ gcc -S hello.i -o hello.s
pravina@LAPTOP-6K8UDRA5:~$ cat hello.s
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Hello, World!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rax
movq %rax, %rdi
call puts@PLT
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
.section .note.GNU-stack,"",@progbits
.section .note.gnu.property,"a"
.align 8
.long 1f - 0f
.long 4f - 1f
.long 5
0:
.string "GNU"
1:
.align 8
.long 0xc0000002
.long 3f - 2f
2:
.long 0x3
3:
.align 8
4:
pravina@LAPTOP-6K8UDRA5:~$
```

Figure 10 : Compilation Stage

Observation: hello.s contains human-readable assembly code, including the main: label and a call to puts to print the message.

Stage 3 : Assembly to Object file

```
pravina@LAPTOP-6K8UDRA5:~/lab2$ rm hello.s
gcc -S hello.i -o hello.s
gcc -c hello.s -o hello.o
file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
pravina@LAPTOP-6K8UDRA5:~/lab2$
```

Figure 11 : Assembly stage

Observation: The file command shows hello.o is an "ELF 64-bit LSB relocatable" file. It contains machine code but cannot run yet, because it has not been linked with the C library.

Stage 4 : Linking

```
pravina@LAPTOP-6K8UDRA5:~/lab2$ gcc hello.o -o hello
./hello
ls -l hello.c hello.i hello.s hello.o hello
Hello, World!
-rwxr-xr-x 1 pravina pravina 15952 Sep 20 14:21 hello
-rw-r--r-- 1 pravina pravina 81 Sep 20 12:45 hello.c
-rw-r--r-- 1 pravina pravina 21435 Sep 20 14:04 hello.i
-rw-r--r-- 1 pravina pravina 1496 Sep 20 14:19 hello.o
-rw-r--r-- 1 pravina pravina 661 Sep 20 14:19 hello.s
pravina@LAPTOP-6K8UDRA5:~/lab2$
```

Figure 12 : Linking Stage

Observation: The linker added the C library , and ./hello printed “Hello, World!”.

Return values and Exit Status

A. Understanding Return Values

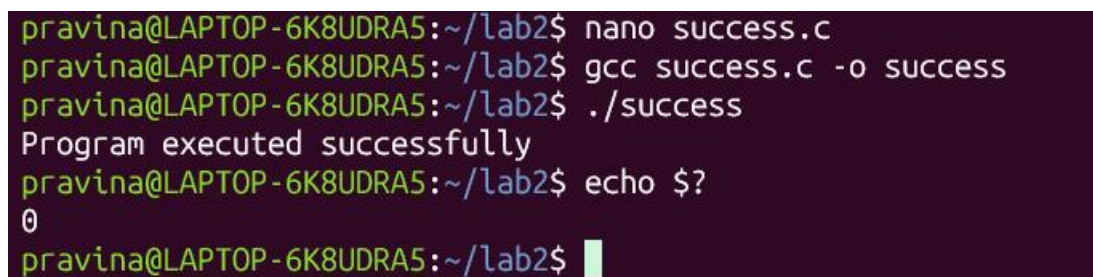
The return value of `main()` tells the operating system whether the program succeeded (0) or failed (non-zero).



```
GNU nano 8.7.1 success.c
#include <stdio.h>

int main() {
    printf("Program executed successfully\n");
    return 0; // 0 indicates success
}
```

Figure 13: Program with return value (success.c)

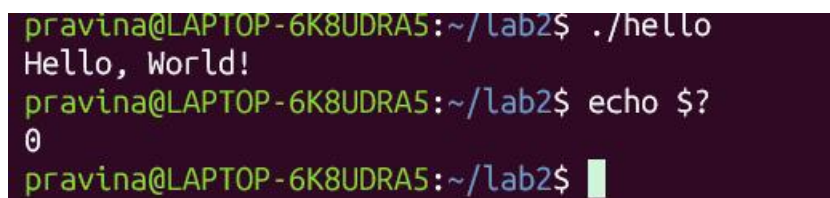


```
pravina@LAPTOP-6K8UDRA5:~/lab2$ nano success.c
pravina@LAPTOP-6K8UDRA5:~/lab2$ gcc success.c -o success
pravina@LAPTOP-6K8UDRA5:~/lab2$ ./success
Program executed successfully
pravina@LAPTOP-6K8UDRA5:~/lab2$ echo $?
0
pravina@LAPTOP-6K8UDRA5:~/lab2$
```

Figure 14: Exit status of success.c is 0 (success)

B. Checking Exit Status

The exit status of a program can be checked with `echo $?` Immediately after running it.



```
pravina@LAPTOP-6K8UDRA5:~/lab2$ ./hello
Hello, World!
pravina@LAPTOP-6K8UDRA5:~/lab2$ echo $?
0
pravina@LAPTOP-6K8UDRA5:~/lab2$
```

Figure 15: Task 1: hello.c returns exit status 0

```
nano hello_error.c
pravina@LAPTOP-6K8UDRA5:~/lab2$ gcc hello_error.c -o hello_error
pravina@LAPTOP-6K8UDRA5:~/lab2$ ./hello_error
Hello, World!
pravina@LAPTOP-6K8UDRA5:~/lab2$ echo $?
1
pravina@LAPTOP-6K8UDRA5:~/lab2$
```

Figure 16 : Task 2: hello_error.c returns exit status 1

KNOWLEDGE CHECK QUESTIONS

Question 1

What is the purpose of the *#include* directive in C programming? Provide an example of its usage.

The *#include* directive imports external files into a program before compilation. It lets you use standard library functions and reuse your own header files instead of rewriting code.

Example:

```
#include <stdio.h>
```

This line includes the Standard Input Output header, which allows the program to use `printf()` and `scanf()`

Question 2

Explain the difference between the four stages of compilation. What file extension is produced at each stage?

The compilation of a C program happens in four separate stages. Each stage produces a different type of output file:

1. **Preprocessing:** Expands *#include* headers and macros, and remove comments. It produces an expanded source code.

Code: `gcc -E <C file (.c)> -o <preprocessed file (.i)>`

2. **Compilation:** Translates the preprocessed C code into assembly language . It produces an assembly code.

Code: `gcc -S <preprocessed file (.i)> -o <assembly code file (.s)>`

3. **Assembling:** Converts the assembly code into machine code producing an object file. It produces an object code.

Code: `gcc -c <assembly code file (.s)> -o <object file (.o)>`

4. **Linking:** Combines the object file with any required libraries and other object files to produce a final, runnable program. It produces an executable file.

Code: `gcc <object file (.o)> -o <executable file>`

`./<executable file>`

Each stage takes the output of the previous one and moves it closer to a program the operating system can actually run.

Question 3

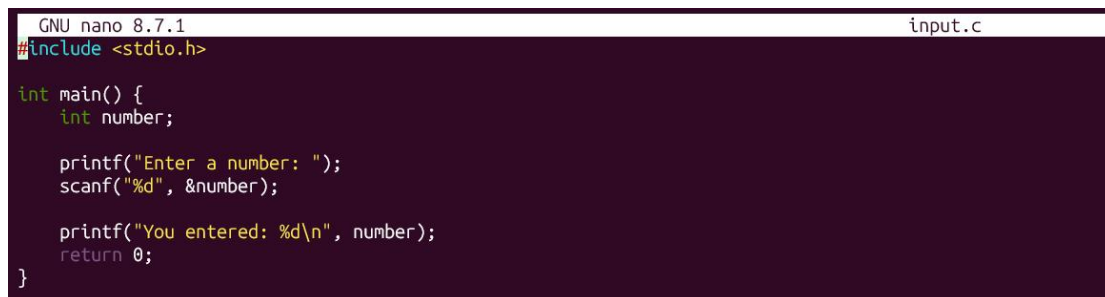
What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

A return value of 0 from main() tells the operating system that the program executed successfully and finished without errors.

A non-zero return value like 1 tells the operating system that the program encountered a problem or ended abnormally. Different non-zero values can be used to represent different types of errors which is useful for scripts that check `echo $?`. It reacts differently depending on the exit code.

Question 4

**Write a C program that:
Declares an integer variable, Asks the user to input a value,
Displays the entered value back to the user.**

A screenshot of a GNU nano 8.7.1 text editor window showing a C program named input.c. The code is as follows:

```
GNU nano 8.7.1 input.c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 17: input.c

Question 5

What GCC command would you use to:

Only preprocess a file without compiling?

Code: gcc -E <C file (.c)> -o <preprocessed file (.i)>

Generate assembly code but not assemble it?

Code: gcc -S <preprocessed file (.i)> -o <assembly code file (.s)>

Create an object file without linking?

Code: gcc -c <assembly code file (.s)> -o <object file (.o)>

Question 6

Explain why C is called a "compiled language" and how this differs from interpreted languages.

C is called a compiled language because its source code is translated entirely into machine code by a compiler before the program runs. The result is an executable file that the computer's processor can run directly, without needing the compiler or source code again.

Interpreted languages, like Python, are different. An interpreter reads and executes the code line by line at runtime, so the source code is needed every time the program runs.

Because of this, compiled programs usually run faster, and errors are caught at compile time. Interpreted programs are usually slower but easier to test and run quickly, since there is no separate compile step.



```
GNU nano 8.7.1 return50.c
#include <stdio.h>

int main() {
    printf("Returning 50\n");
    return 50;
}
```

Figure 18 : return50.c

```
pravina@LAPTOP-6K8UDRA5:~/lab2$ nano return50.c
pravina@LAPTOP-6K8UDRA5:~/lab2$ gcc return50.c -o return50
pravina@LAPTOP-6K8UDRA5:~/lab2$ ./return50
Returning 50
pravina@LAPTOP-6K8UDRA5:~/lab2$ echo $?
50
pravina@LAPTOP-6K8UDRA5:~/lab2$ █
```

Figure 19 : Return value

CONCLUSION

This lab provides practical experience with the fundamentals of C as shown in the linux system as shown above. The code is also run and written. We have also separately checked the role of preprocessor directives, the main() function, and the standard input and output functions. The four compilations stages were also carried out. The stages show how machine code is turned into executable files which the operating system can load and run. Testing different return values shows how a program communicates success or failure to the shell. A return value of 0 means it is successful and non-zero values like 1 and 50 are spotted by echo \$? . This confirms that the exit status is passed from the program to the operating system.

In conclusion, the lab objectives were met, that is to understand the compilation pipeline and exit codes.

They are essential background for the systems programming, process management, and security topics covered later in this module

GITHUB REPOSITORY

[Github Repo](#)

Appendix

```
pravina@LAPTOP-6K8UDRA5:~/lab2$ cd ~/lab2
pravina@LAPTOP-6K8UDRA5:~/lab2$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
```

```
pravina@LAPTOP-6K8UDRA5:~/lab2$ git config --global user.name "Prabina-1"
pravina@LAPTOP-6K8UDRA5:~/lab2$ git config --global user.email "xaudharyprabina@gmail.com"
pravina@LAPTOP-6K8UDRA5:~/lab2$ git commit -m "Lab 2: C programming basics"
[master (root-commit) b945645] Lab 2: C programming basics
17 files changed, 958 insertions(+)
create mode 100755 hello
create mode 100644 hello.c
create mode 100644 hello.i
create mode 100644 hello.o
create mode 100644 hello.s
create mode 100755 hello_error
create mode 100644 hello_error.c
create mode 100755 input
create mode 100644 input.c
create mode 100755 return50
create mode 100644 return50.c
create mode 100755 success
create mode 100644 success.c
create mode 100755 task1/task1
create mode 100644 task1/task1.c
create mode 100755 userinfo
create mode 100644 userinfo.c
```

```
Username for 'https://github.com': prabina-1
Password for 'https://prabina-1@github.com':
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 12 threads
Compressing objects: 100% (20/20), done.
Writing objects: 100% (20/20), 13.27 KiB | 4.42 MiB/s, done.
Total 20 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), done.
To https://github.com/Prabina-1/os-labs.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
pravina@LAPTOP-6K8UDRA5:~/lab2$
```

